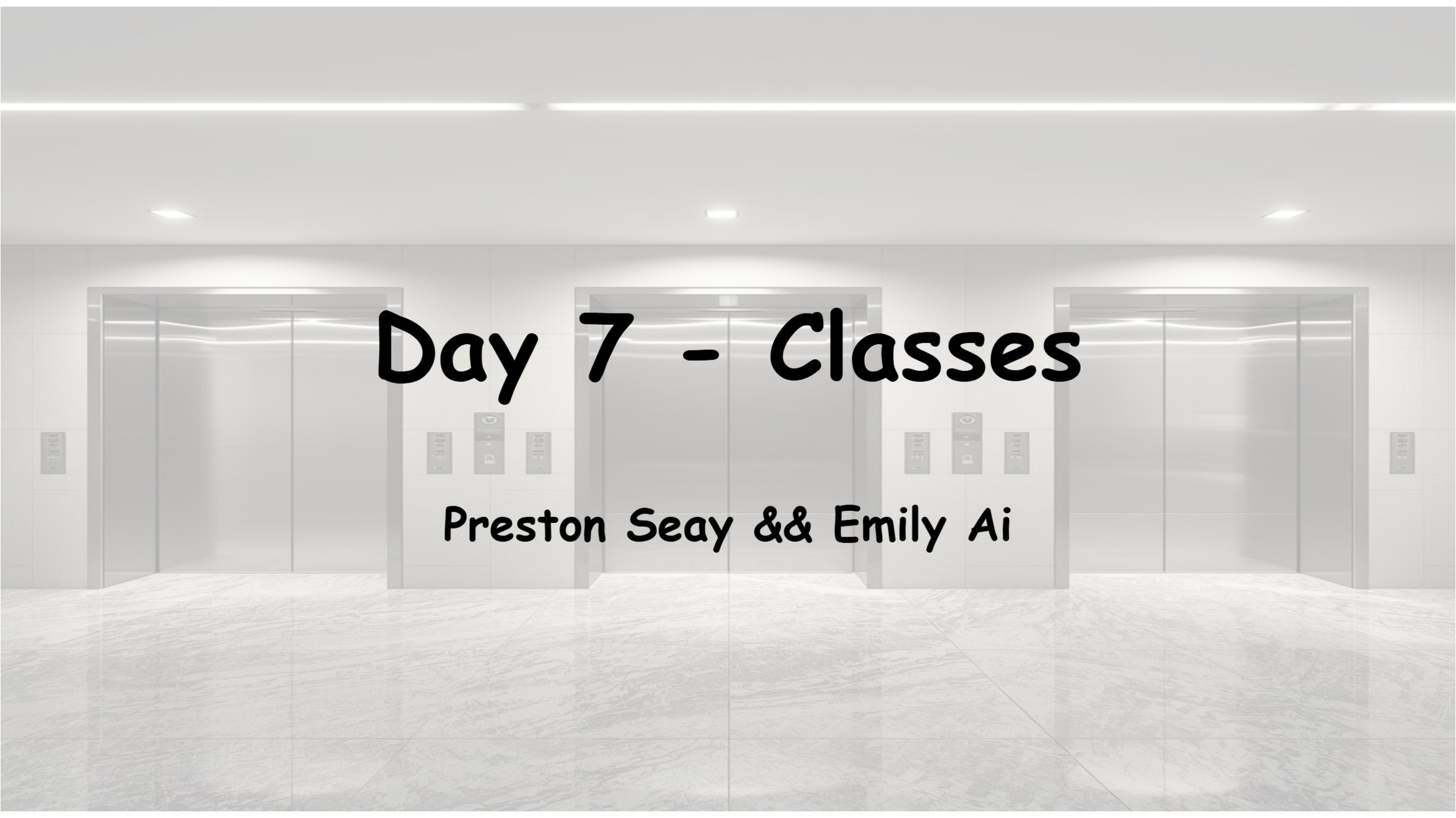
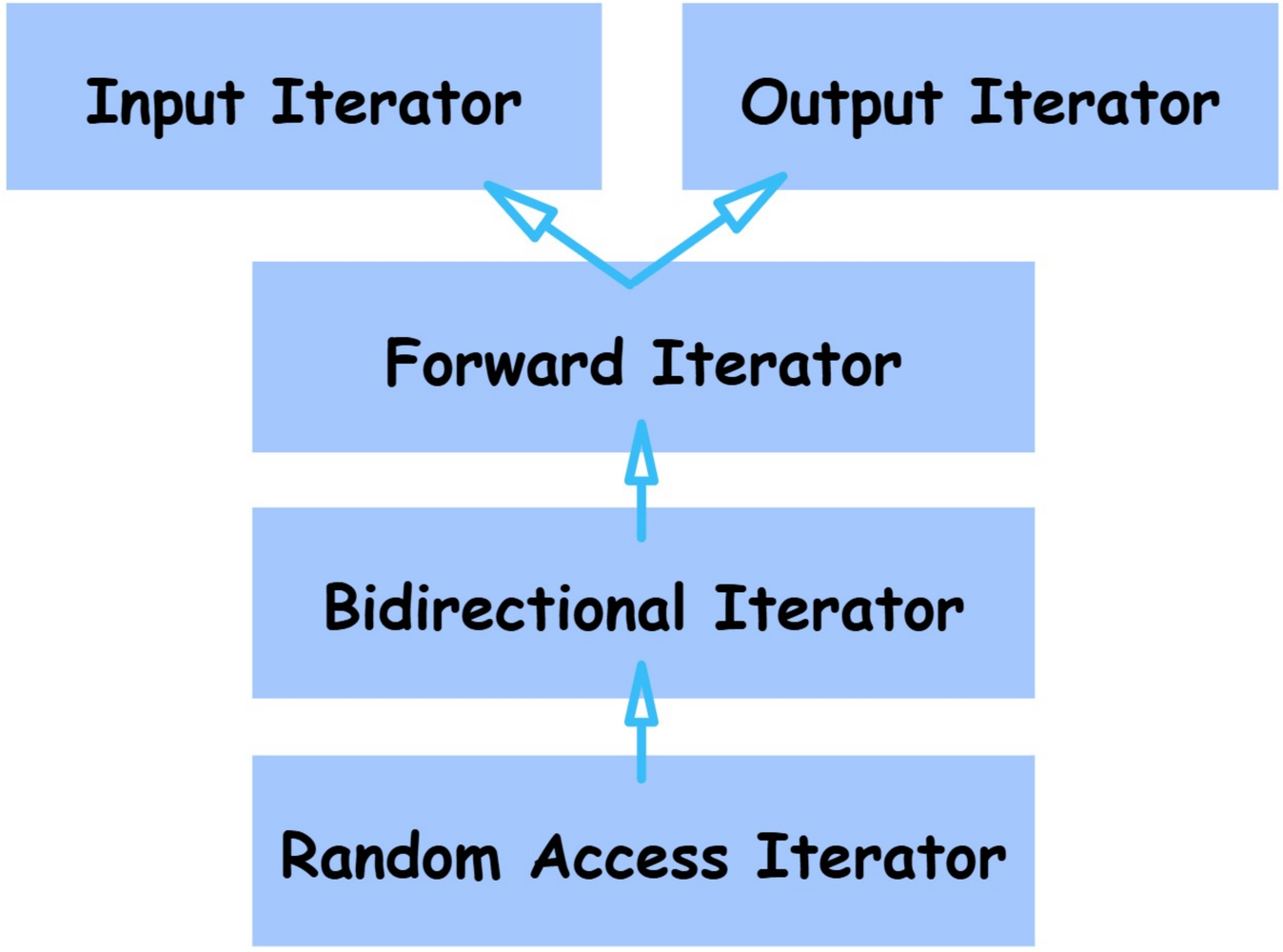


Day 7 - Classes

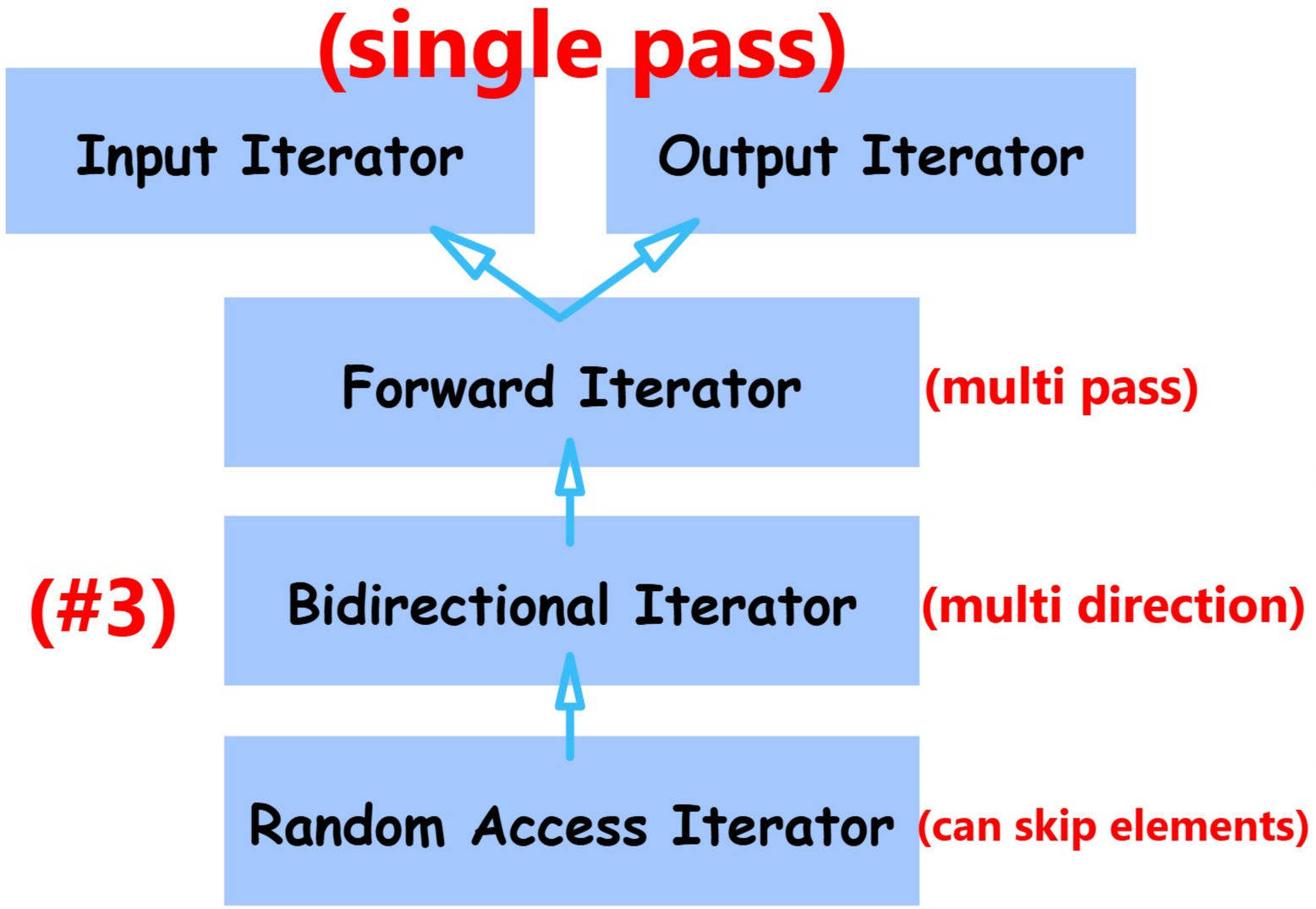
Preston Seay && Emily Ai

Iterator Recap



1. What can you do with a Bidirectional Iterator that you cannot do with an Input Iterator?
2. What can you do with a Random Access Iterator that you cannot do with a Bidirectional Iterator?
3. What type of iterator is std::set<int>::iterator?

Iterator Recap



1. What can you do with a Bidirectional Iterator that you cannot do with an Input Iterator?
2. What can you do with a Random Access Iterator that you cannot do with a Bidirectional Iterator?
3. What type of iterator is std::set<int>::iterator?

Attendance



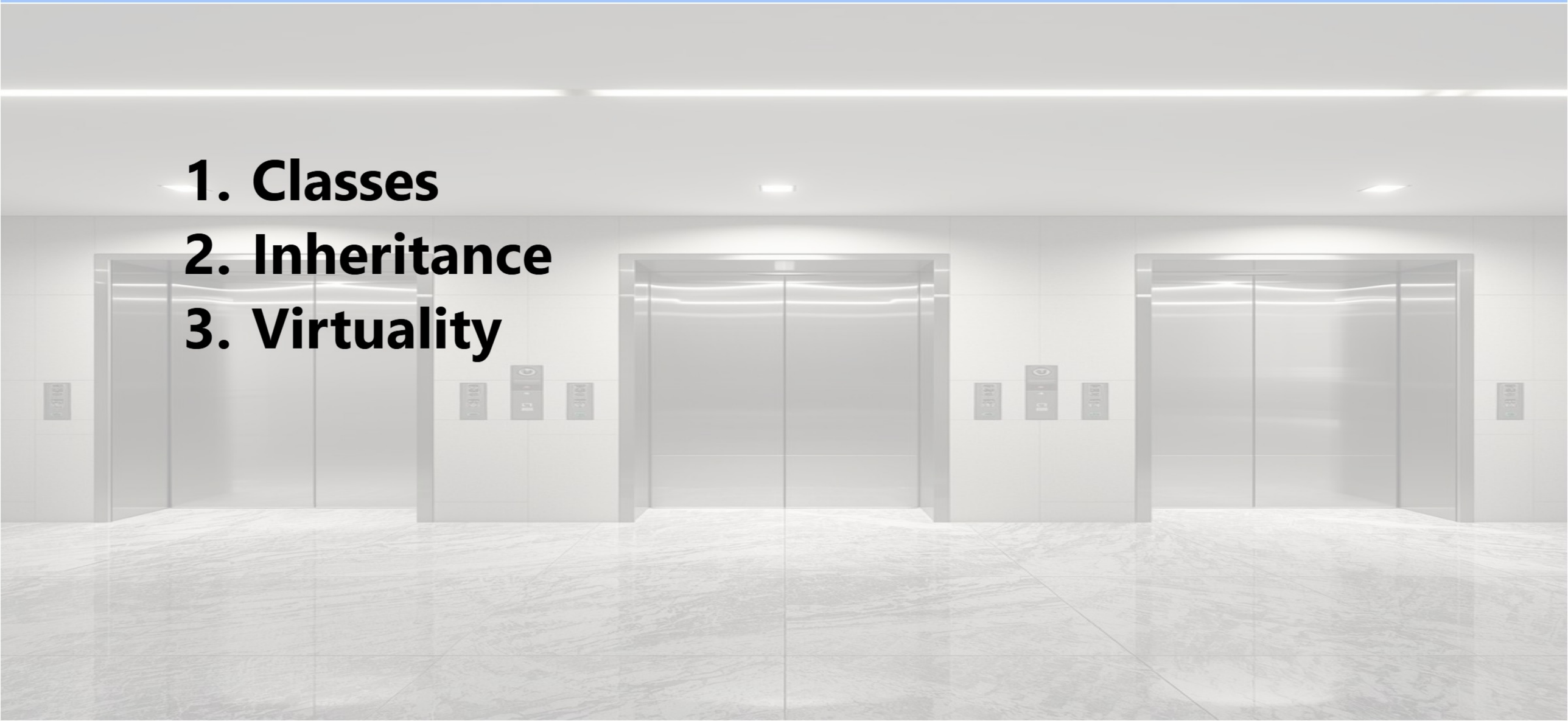
https://www.106l.com/live/classes/warm-up?key=L20vGxrD9c_aQv5uJOu2qynn7OPt3s_ZoVAKrQELHKs

Reminder

Attendance is required.

If you've missed lecture, you have makeup work due.

Plan



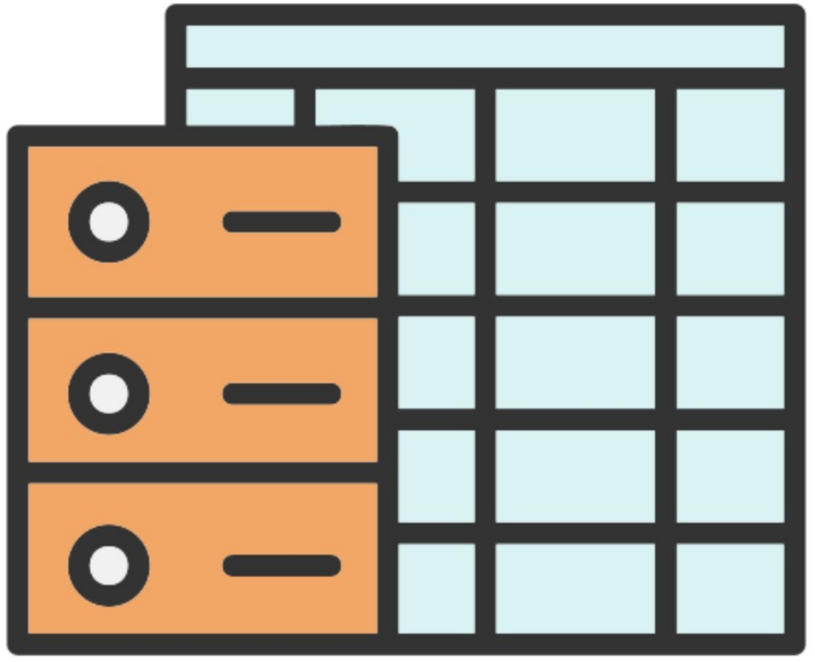
1. Classes
2. Inheritance
3. Virtuality

Why Classes

C



C++



C++ was originally called “C with classes.”

Why Classes

Scary Underlying Data



Clean Interface

...

`erase`	erases elements (public member function)
`push_back`	adds an element to the end (public member function)

...

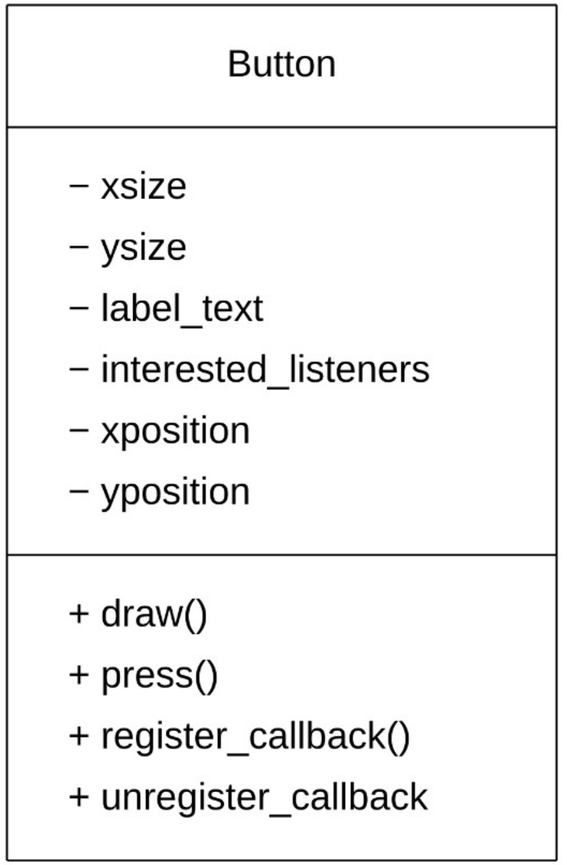
Abstraction Boundary (Encapsulation)

Why Classes

Object Oriented Programming (OOP)

Data is grouped in objects.

Objects contain members: fields and functions.



What Classes

A class is a user-defined type.




```
Cookie c1;
```

```
Cookie c2;
```

Instances

```
class Cookie {};
```

(Data) Type

How Classes

We've seen these before?! Aren't `structs` user-defined types?

```
1 struct StanfordID {
2   // these are fields
3   std::string name;
4   std::string sunet;
5   int idNumber;
6 };
```

```
1 StanfordID id;
2 id.name = "Preston";
3 id.sunet = "pseay";
4 id.idNumber = 100;
5 id.idNumber = -100;
```

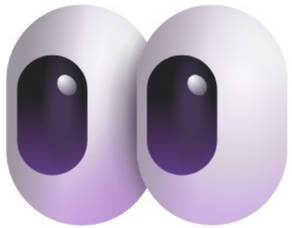


No access controls

Classes: Access Controls

```
1  class CoolClass {
2  private:
3
4
5
6
7  public:
8
9
10
11
12 };
```





You can define public and private sections.

Classes: Access Controls

Structs



Classes



Activity: Set the right access specifier.

106l.com slide 2

1 Class, 2 Files

	Header File (.h)	**Source File (.cpp)**
Purpose	Defines the interface	Implements class functions
Contains	Function prototypes, class declarations, type definitions, macros, constants	Function implementations, executable code
Access	This is shared across source files	Is compiled into an object file
Example	`void someFunction();`	`void someFunction() {...};`

1 Class, 2 Members

- **Data** members (sometimes "fields" or "member variables")
- Member **functions** (sometimes "methods")
    - Special: Constructor, Destructor, etc.

1 Class, 1 Example

```h
// File: elevator.h

class Elevator {
private:
  // Private member fields.
  int floor;
  int capacity;
public:
  // Constructor.
  Elevator(int capacity);
  // Public member functions.
  void request(int floor);
};
```

```cpp
// File: elevator.cpp
#include "elevator.h"

Elevator::Elevator(int capacity) {
  this->capacity = capacity;
  floor = 0;
}

void Elevator::request(int floor) {
  this->floor = floor;
}
```

1 Class, 1 Example

```
1  // File: elevator.h
2
3  class Elevator {
4  private:
5    // Private member fields.
6    int floor;
7    int capacity;
8  public:
9    // Constructor.
10   Elevator(int capacity);
11   // Public member functions.
12   void request(int floor);
13 };
```

```
1  // File: elevator.cpp
2  #include "elevator.h"
3
4  Elevator::Elevator(int capacity) {
5    if (capacity < 0) {
6      cerr << "Invalid capacity!\n";
7      capacity = 1;
8    }
9
10   this->capacity = capacity;
11   floor = 0;
12 }
13
14 void Elevator::request(int floor) {
15   this->floor = floor;
16 }
```

1 Class, 1 Example

```cpp
// File: main.cpp
#include "elevator.h"

int main() {
    Elevator e { 1 };
    e.request(1);
    return 0;
}
```

1 Class, 1 Example

```
1  // File: elevator.h
2
3  class Elevator {
4    // ...
5    int capacity;
6  public:
7    // ...
8    // Private members often have "getters."
9    int getCapacity(int capacity);
10   // They also often have "setters."
11   void setCapacity(int floor);
12 };
```

Getters, Setters, Confusers



Activity: Implement getters & setters.

106l.com slide 3

Classes: Constructors & Destructors

```
1  // File: elevator.h
2
3  class Elevator {
4  private:
5      int[] floors;
6  public:
7      // Constructor:
8      Elevator(int numFloors);
9      // Destructor:
10     ~Elevator();
11 };
```

```
1  // File: elevator.cpp
2  #include "elevator.h"
3
4  Elevator::Elevator(int numFloors) {
5      this->floors = new int[numFloors];
6  }
7
8  Elevator::~Elevator() {
9      delete[] this->floors;
10 }
```

Bonus: Initializer List Constructor

```h
// File: elevator.h

class Elevator {
private:
  int[] floors;
public:
  // Constructor:
  Elevator(int numFloors);
  // Destructor:
  ~Elevator();
};
```

```cpp
// File: elevator.cpp
#include "elevator.h"

Elevator::Elevator(int numFloors) :
  floors { new int[numFloors] }
{

}

Elevator::~Elevator() {
  delete[] this->floors;
}
```

Activity: Implement constructor & destructor?

Plan

- Classes
  - Access Specifiers
  - Data Members
  - Member Functions
    - Constructor & Destructor
- **Inheritance**
- Virtuality

Inheritance







Inheritance

Elevator:
- Capacity
- Floors

IkeaElevator



ScaryElevator



WaterElevator



Inheritance

Vehicle
+ goTo(destination)

Elevator:
- Capacity
- Floors

Car:
- Seats
- Mileage

InNOutMobile:
+ serveFries()



Inheritance

- **Dynamic Polymorphism:** Having the type be determined at runtime.
- **Extensibility:** Adding more to a class.

Inheritance: How to...

```
1  // shapes.h
2  class Rectangle {
3  private:
4      int width;
5      int height;
6  public:
7      Rectangle(int w, int h);
8      int getArea();
9  };
10
11 class Square : Rectangle {
12     Square(int len);
13 };
```

```
1  // shapes.cpp
2  #include "shapes.h"
3
4  Square(int len) :
5      Rectangle(len, len) {}
```

But... is **getArea** public or private on Square?

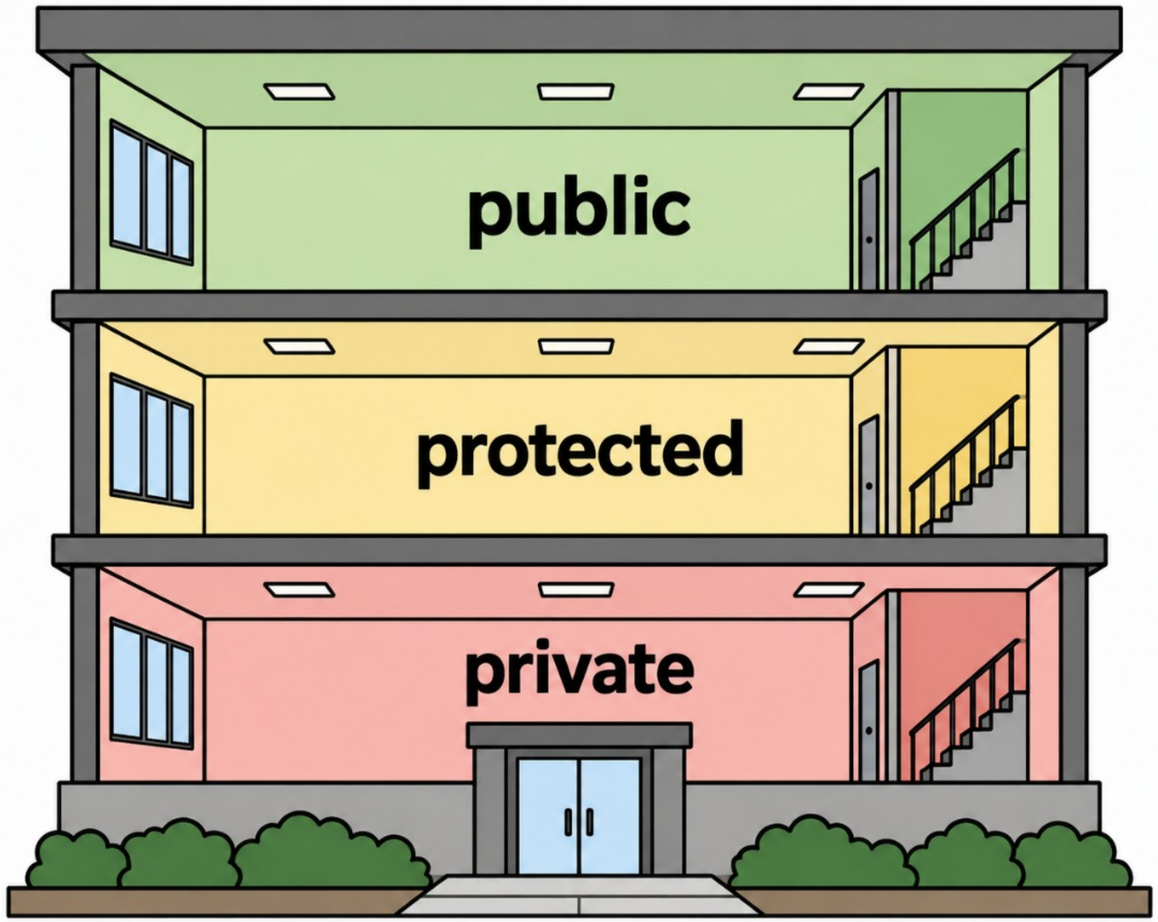
Inheritance Access Specifiers

```
1  // shapes.h
2  class Rectangle {
3  private:
4      int width;
5      int height;
6  public:
7      Rectangle(int w, int h);
8      int getArea();
9  };
10
11 class Square : public Rectangle {
12     Square(int len);
13 };
```

private (default for classes) - all members of parent are private.
public - all public members of the parent are public.

But… is `getArea` public or private on Square?

One more thing: protected

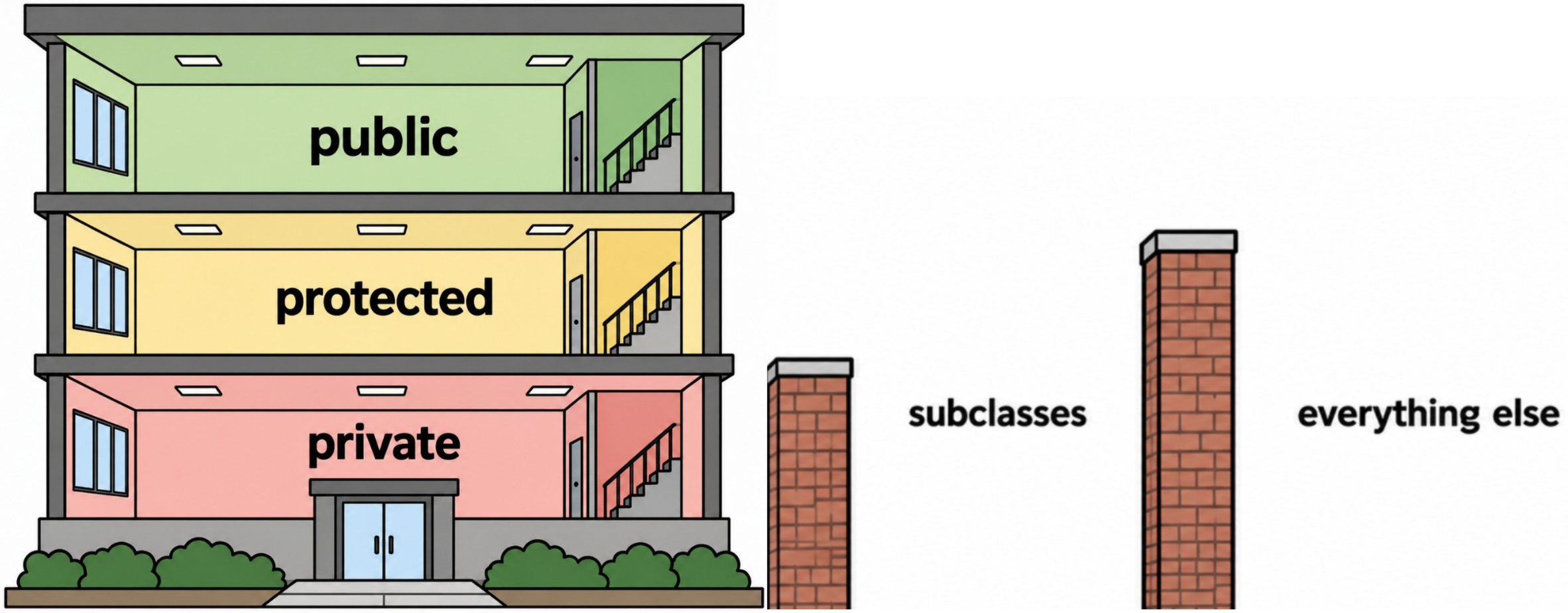


Anyone can see/modify.

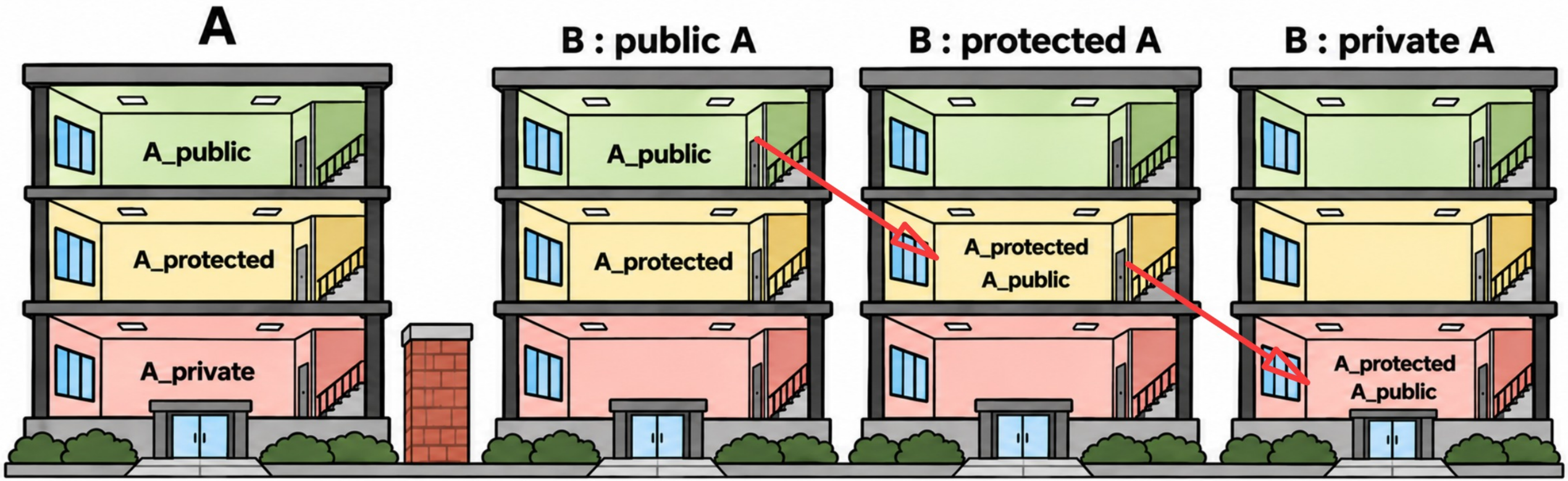
Subclasses can see/modify.

Nobody else can see/modify.

Access Specifiers



Access Specifiers



Activity: Choose the right type of inheritance.

Plan

Classes

Inheritance

- public, protected, private
- initializer list constructor

Virtuality

Abstract Classes

```
1  // vehicles.h
2
3  class Vehicle {
4  protected:
5      int seats;
6  public:
7      Vehicle(int seats);
8      virtual void getCapacity() = 0;
9  };
10
11 class Bus : public Vehicle {
12     void getCapacity() override;
13 };
```

Abstract classes: Classes that cannot be initialized.

Pure functions: Functions that do not have an implementation.

`override` Tells the compiler that this must match a virtual function of a parent class.

Abstract Classes

```cpp
// vehicles.h

class Vehicle {
protected:
  int seats;
public:
  Vehicle(int seats);
  virtual int getCapacity() = 0;
};
class Bus : public Vehicle {
  Bus(int seats);
  int getCapacity() override;
};
```

```cpp
// vehicles.cpp

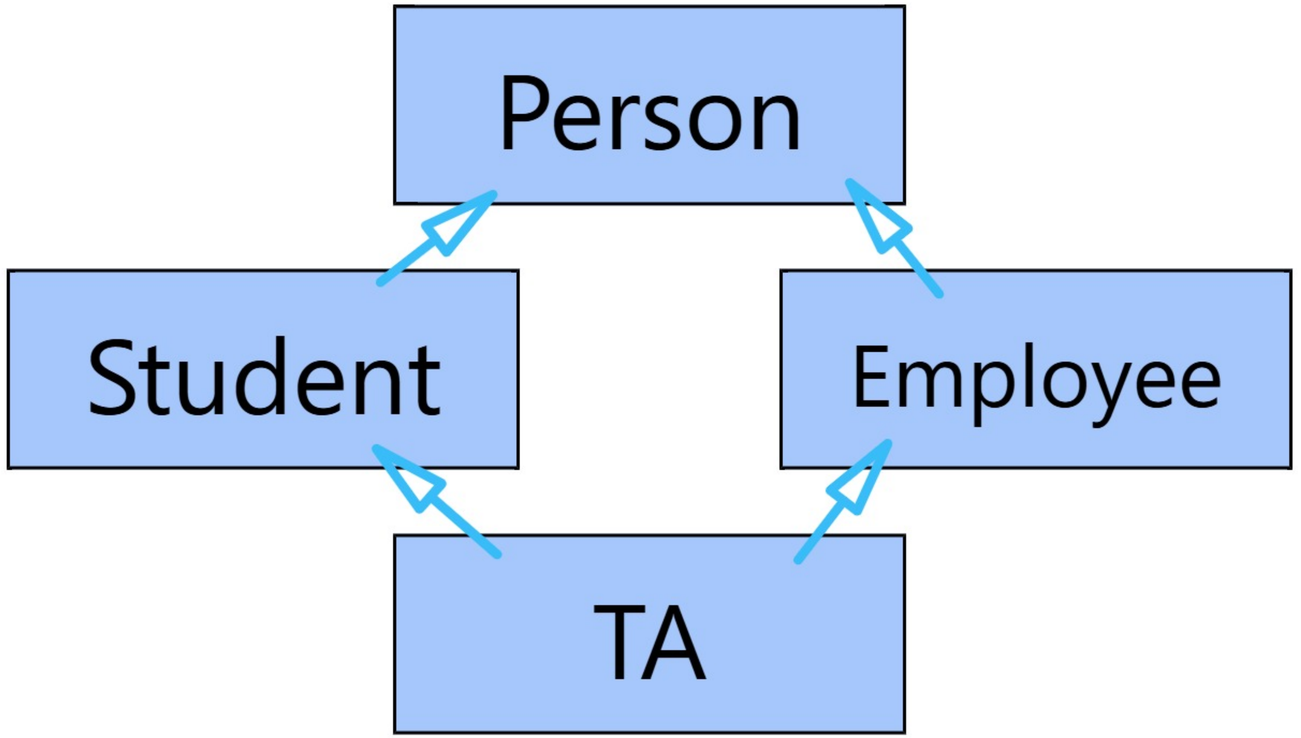
Bus::Bus(int seats) :
  Vehicle { seats } {}

int Bus::getCapacity() override {
  return seats * 2;
}
```

Diamond Problem

```
1  // vehicles.h
2  struct Person {
3      std::string name;
4  };
5  struct Student : Person {}
6  struct Employee : Person {}
7  struct TA : Student, Employee {}
8
9  TA me;
10 me.name = "Preston";
```

ERROR

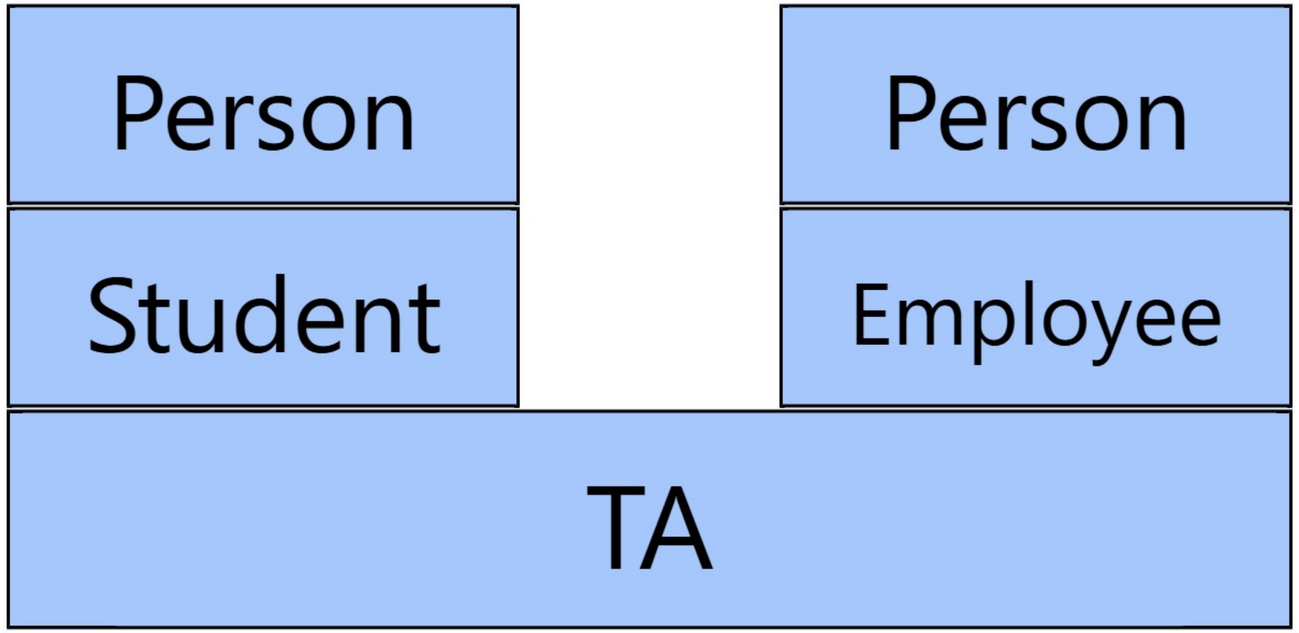


A TA Inherits 2 Persons.

Diamond Problem

```
1  // vehicles.h
2  struct Person {
3      std::string name;
4  };
5  struct Student : Person {}
6  struct Employee : Person {}
7  struct TA : Student, Employee {}
8
9  TA me;
10 me.name = "Preston";
```

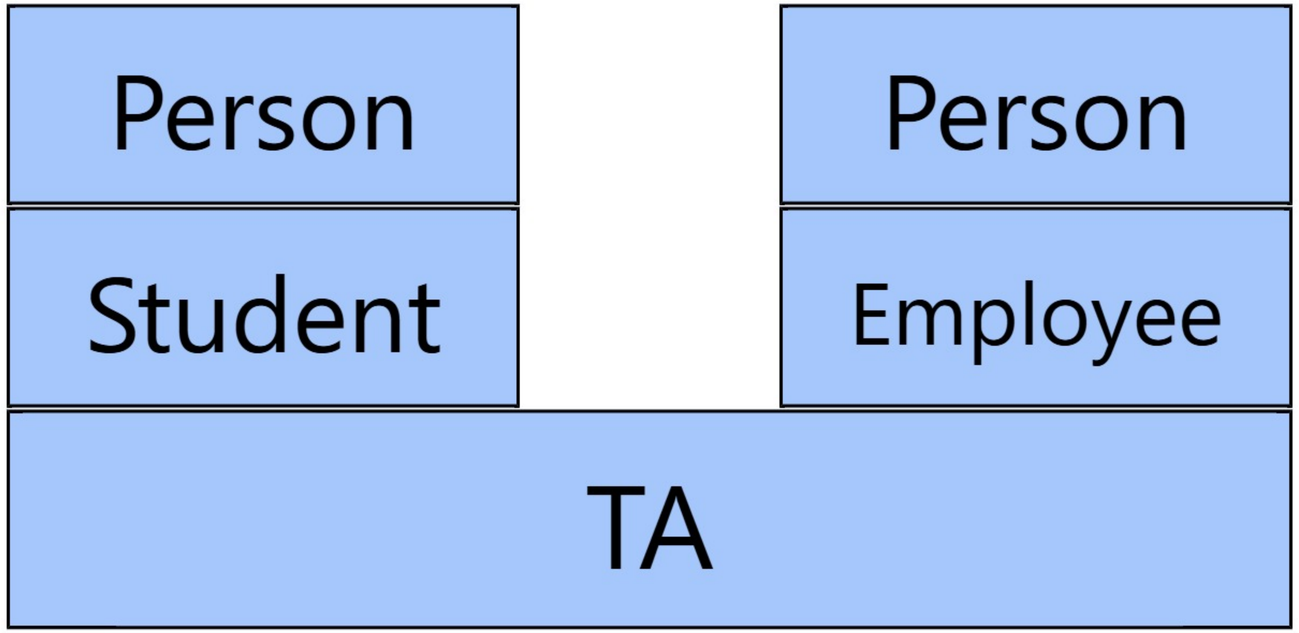
ERROR



A TA Inherits 2 Persons.

Diamond Solution #1

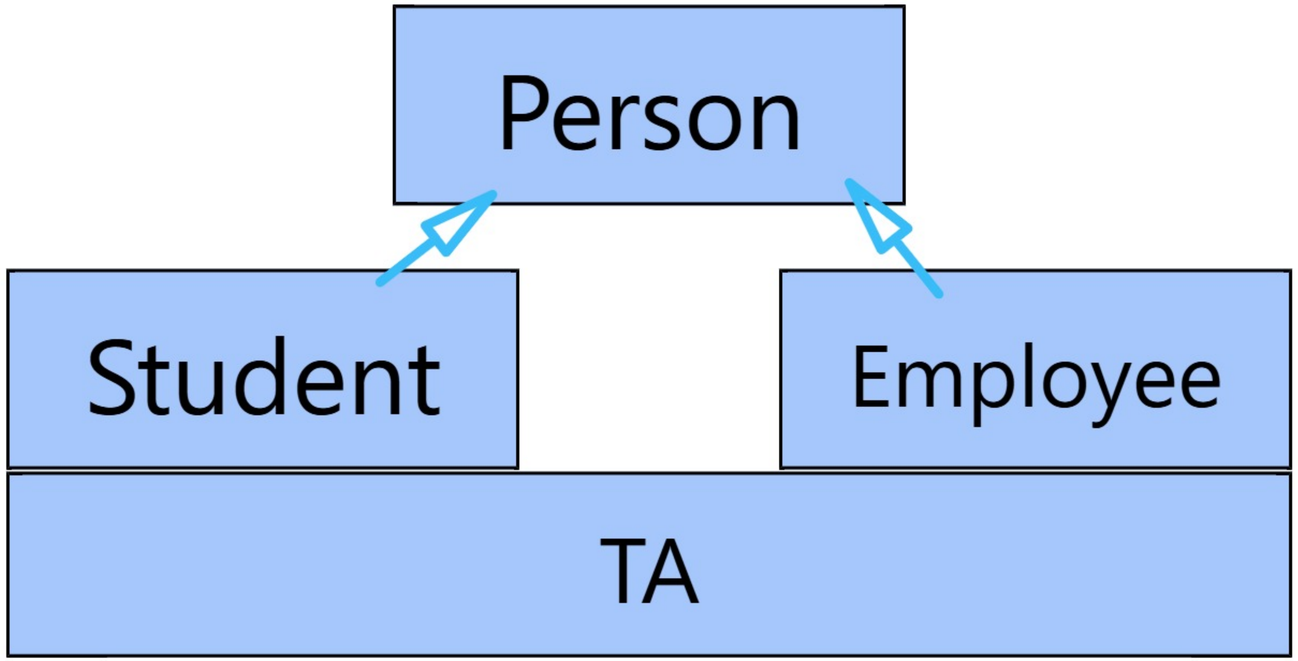
```
1  // vehicles.h
2  struct Person {
3      std::string name;
4  };
5  struct Student : Person {}
6  struct Employee : Person {}
7  struct TA : Student, Employee {}
8
9  TA me;
10 me::Student.name = "Preston";
11 me::Employee.name = "Mr. Seay";
```



Specify which **Person**.

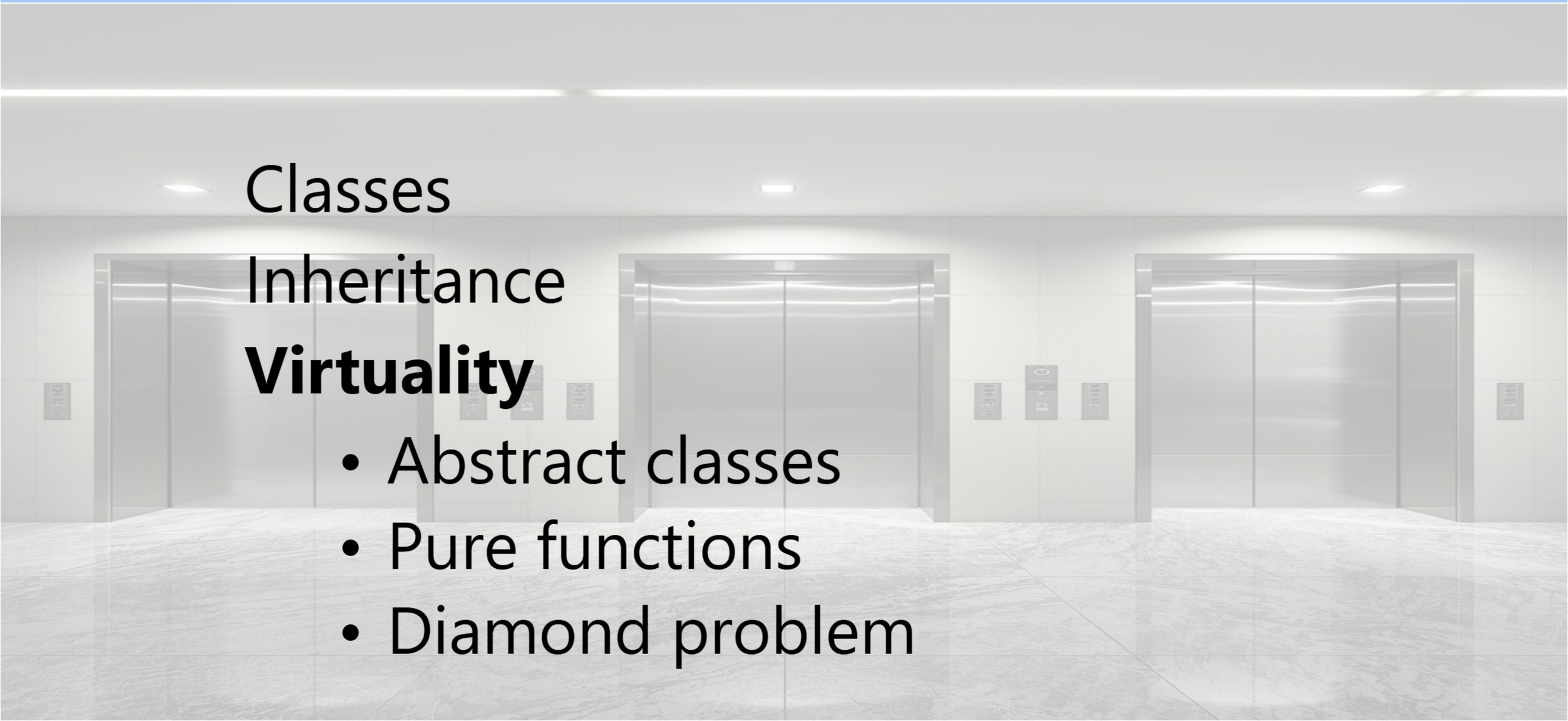
Diamond Problem

```
1  // vehicles.h
2  struct Person {
3      std::string name;
4  };
5  struct Student : virtual Person {}
6  struct Employee : virtual Person {}
7  struct TA : Student, Employee {}
8
9  TA me;
10 me.name = "Preston";
```



Virtually Inherit **`Person`**, so there's just 1.

Plan



Classes

Inheritance

Virtuality

- Abstract classes
- Pure functions
- Diamond problem

Recap

- We can make types with `class`.
- Encapsulate variables with limited access specifiers.
- Inheritance let's us reuse and extend functionality.
- `virtual` gives us control of inheritance.

Complete the Activity:

Work with others and ask questions.

